Birla Institute of Technology & Science, Pilani

Work Integrated Learning Programmes Division

First Semester 2026

Assignment

(EC-1)

Course No. : SE*ZG504

Course Title : API-based Products

Weightage : 30%

Duration : 2 Weeks

No. of Pages = 4

Note to Students:

1. Please ensure you capture the screenshots to demonstrate that you have performed the assignment.
2. Each question carriers 10 marks. Total 30 Marks.
3. Upload a single ZIP file with PDF file with all the necessary screenshots and code snippets. Provide instructions to compile and run the code.

Question 1

Build a **gRPC-based Task Manager service** that allows clients to:

1. **Create a new task**
2. **Fetch a task by ID**
3. **List all tasks** (server-streaming)
4. **Update task status**
5. **Stream task updates** (bidirectional streaming)

A *Task* contains:

- id (integer)
- title (string)
- status (string: "pending", "in_progress", "done")

You must:

Define a .proto file containing:

- Task message
- CreateTaskRequest

- GetTaskRequest
- UpdateTaskRequest
- TaskService with the RPCs that meet the above 5 requirements.

Implement the server in **any programming language (Python/Java/C#)**.

Your server must:

- Store tasks in memory
- Generate incremental task IDs
- Support server-streaming for ListTasks
- Support bidirectional streaming for StreamTaskUpdates
- Validate that status is one of: pending, in_progress, done
- Return an error if the task ID does not exist

Implement a client that:

1. Creates three tasks
2. Fetches task #2
3. Streams all tasks using ListTasks
4. Sends three updates using StreamTaskUpdates and prints server responses

Add the following error cases:

- If GetTaskRequest.id < 1, return INVALID_ARGUMENT
- If task not found, return NOT_FOUND
- If status is invalid, return FAILED_PRECONDITION

Modify the client to call GetTask with a **1-second deadline**.

If the server sleeps for 2 seconds before responding, the client must handle the timeout gracefully.

Question 2

You are building a simple **Order API** that must be callable by an LLM through **Model Context Protocol (MCP)**. The MCP server will expose the API as a **tool** that the LLM can invoke. Finally, the API must be published through **Kong API Gateway**, where **rate limiting** must be applied.

Your task is to build:

1. A REST API
2. An MCP server that exposes the API as a tool
3. Kong configuration to apply rate limiting

Flow:

LLM → MCP Client → MCP Server → Your API → Kong Gateway → API Backend

PART A — Build the Order API

Create a REST API with the following endpoints:

```
POST /orders

GET /orders/{id}

GET /orders
```

Requirements:

- Store orders in memory
- Each order must contain:
    - id (integer)
    - item (string)
    - quantity (integer)
    - status (string: "created")
- Validate: quantity > 0
- Return proper HTTP error codes
- Use JSON for all input/output

PART B — Expose the API as an MCP Tool

Create an MCP server that exposes the following tool:

Tool Name: order_api

Tool Actions:

1. create_order : calls POST /orders
2. get_order : calls GET /orders/{id}
3. list_orders : calls GET /orders

Your MCP server must:

- Validate the input schema
- Route the call to the correct API endpoint
- Return the API's JSON response

PART C — Publish the API Through Kong

Use Kong to expose the API.

You must:

1. Register the API as a Kong service
2. Create a Kong route /api/orders
3. Apply Kong's **rate limiting plugin**
4. Set the limit to **5 requests per minute**
5. Demonstrate rate limiting by sending more than 5 requests

PART D — Deliverables

Your submission must include:

1. API source code
2. MCP server source code
3. MCP tool schema
4. Kong configuration (Admin API or declarative YAML)
5. Screenshot/log showing rate limiting

Question 3

Implement TokenBucket and Leaky Bucket rate limiting algorithms in Java/Python/c#.